

Homework 1

by Petros Efraim Koukios, and Alex Rysström
Group 68

January 27, 2026

Question 1: Compute the Sum, Min, and Max of Matrix Elements

In this problem, pthreads were used to concurrently find the max, min, and sum of matrix elements. The question was split into three implementations consisting of:

- An implementation using a barrier and arrays for partial results where worker 0 does the final computations and printing.
- An implementation removing the barrier and partial result arrays. Here the main thread does not retire and does the final computations and outputs.
- An implementation using a bag of tasks.

This report will cover some of the main ideas used in the solutions.

In the first implementation, the given file "matrixSum.c" was used as a template. Very little was changed in the flow and logic of the program. The main adaptation involved adding more partial results arrays to store all the different results (sum, min, max, min indices, max indices), as shown in the following code snippet (taken from the **Worker** function):

```
/* storing results in partial result arrays */
sums[myid] = total;
min_vals[myid] = min_val;
max_vals[myid] = max_val;
min_row_index[myid] = min_row;
min_col_index[myid] = min_col;
max_row_index[myid] = max_row;
max_col_index[myid] = max_col;
```

```
Barrier(); //all threads call barrier to sync
```

```
/*after barrier only worker 0 does computations and prints*/
```

In the second implementation, the `pthread_create` sends the workers out and instead of exiting immediately, the main thread waits. Each worker allocates a `ThreadResult` struct in memory and writes its results into this struct. When a worker retires, it calls `pthread_exit` and returns the address of the results. The final processing of worker results are done in the main thread rather than by worker 0. The struct is as shown below:

```
/*struct named ThreadResult to return data from threads*/
typedef struct {
    long long total;
    int max_val;
    int min_val;
    int max_row, max_col;
    int min_row, min_col;
} ThreadResult;
```

Part C was adapted from the previous part to include a bag of tasks. Instead of assigning strips of rows beforehand, threads atomically retrieve row indices from a shared counter protected by a mutex.

```
while (true) {
    /*lock when access bag*/
    pthread_mutex_lock(&row_lock);
    current_row = next_row; //get task from bag
    next_row++;             //increment counter
    pthread_mutex_unlock(&row_lock);
    /*critical section end*/

    //check if bag is empty
    if (current_row >= size) {
        break;
    }

    //process specific row retrieved from the bag
}
```

Other small changes were made from the original "matrixSum.c" and carried over through the sub problems such as adjusting the type for **total** to be *long long* to account for overflow when summing thousands of elements. Additionally, the matrix initialisation was changed to randomly generate integers in each position rather than being a matrix consisting of purely 1's.

Question 2. Quicksort

This program implements a parallel version of the quicksort algorithm using POSIX threads. Quicksort is an algorithm that sorts an array by selecting a pivot element, partitioning the array into two parts around the pivot, and then recursively sorting the resulting subarrays.

The last element was used as the pivot, which is a valid and simple strategy but may lead to unbalanced partitions in some cases, potentially decreasing performance to $O(n^2)$. The average time complexity remains $O(n \log n)$, which is okay for this implementation.

```
int pivot = arr[high];
```

The partition function rearranges the elements so that all values less than or equal to the pivot are placed to the left, while all values greater than the pivot are placed to the right. After partitioning, the pivot is placed in its final sorted position, and the algorithm continues recursively on the left and right subarrays.

Parallelism is achieved by creating recursive threads. When a partition is created, a new thread may be created to sort one subarray while the current thread sorts the other. This behavior is implemented using `pthread_create()` and recursive calls:

```
pthread_create(&t, NULL, quicksort_parallel, &left);
quicksort_parallel(&right);
```

A max recursion depth parameter is used to limit the number of threads created, preventing excessive thread creation and reducing overhead:

```
if (a->depth < MAX_THREADS)
```

In addition, a hybrid approach is used where small subarrays are sorted sequentially instead of creating new threads, which reduces thread management overhead while preserving correctness:

```
if ((a->high - a->low) < MIN_SIZE) {
    quicksort_seq(a->array, a->low, a->high);
}
```

A shared counter is used to track the number of active worker threads. Access to this counter is protected by a mutex to ensure mutual exclusion and prevent race conditions:

```
pthread_mutex_lock(&count_mutex);
active_threads++;
pthread_mutex_unlock(&count_mutex);
```

A condition variable allows the main thread to wait until all worker threads have completed execution. When the last active thread finishes, it signals the condition variable so that the main thread can proceed:

```
pthread_cond_wait(&done_cond, &count_mutex);
```

The execution time of the program is measured using the `gettimeofday()` function. Timing begins just before the initial sorting thread is created and ends after all threads have completed execution.

Performance Evaluation

The performance results show that the execution time increases as the input size grows, which is expected for a quicksort-based algorithm. For small input sizes, execution time remains relatively constant due to thread creation, synchronization overhead, and timer resolution effects dominating the runtime. As the input size increases, the execution time grows more noticeably and follows the expected average-case time complexity of $O(n \log n)$ for quicksort. Compared to a non-parallel quicksort, the parallel implementation improves performance for larger input sizes by allowing multiple subarrays to be processed concurrently on different cores. The use of a sequential fallback for small subarrays helps reduce unnecessary thread overhead, resulting in more stable performance across different input sizes.

Input size (n)	Execution time (seconds)
1000	0.001333
2000	0.002329
4000	0.004620
8000	0.004627
16000	0.006030
32000	0.008873
64000	0.016089
128000	0.022197
256000	0.033306
512000	0.058855
1024000	0.132090
2048000	0.251689
4096000	0.403548
8192000	1.193600

